

HR Analytics Project: Solutions Report

Group Details

GROUP NUMBER: 18

GROUP MEMBERS DETAILS:

	NAME	SURNAME	STUDENT NUMBER
1	Portia Sebatso	Thulo	2923783
2	Katlego	Sebopa	2465165
3	Zodwa	Mtambo	2625574
4	Azasakhe	Bell	1907146
5	Ntando	Somthunzi	2775317

Introduction

This report details our approach on coming up with functions that identify and fix issues in the HR dataset (uncleaned dataset), and then use the cleaned data to calculate key HR metrics.

The main focus is on how we came up with our solutions using Python data structures such as lists, dictionaries, and sets, and how we applied loops and conditional logic to clean and analyse the dataset.

Section 1: Data Cleaning

1. Remove Null Salaries

The issue here was missing or invalid salary values (NaN). This is a problem because salary is a numeric field used in later calculations, so any missing or incorrect values would affect the accuracy of results.

To fix this, we first created an empty list called `cleaned_data` to store only employee records with valid salaries. We then used a for loop to iterate through each record in the dataset.

Inside the loop, we accessed the salary at index 4 and checked if it was invalid, that is if it was:

- missing (')
- not a number (`float('nan')` or `salary!=salary`)

If the salary was invalid, we skipped the record using `continue`. Otherwise, we added (appended) the record into `cleaned_data`. The function returns this `cleaned_data` list containing records with valid salary entries.

2. Standardize Departments

The issue was inconsistent department formatting (uppercase vs lowercase). This is a problem because Python is case-sensitive, so the same department could be treated as different values during analysis.

To fix this, we used a for loop to iterate through each employee record and accessed the department at index 1. We then applied `.lower()` to standardise all values to lowercase.

3. Removing Invalid Performance Ratings

The issue was the presence of performance ratings that were either outside the valid range (0 to 5) or invalid values. This would distort analysis results if not removed.

To fix this, we created a new empty list called `cleaned_data` to store valid records only. We then used a for loop to iterate through the dataset.

For each record, we accessed performance rating at index 6 and checked if it was invalid, that is if it was:

- less than 0
- greater than 5
- or invalid (`float('nan')`)

If the rating for a record was invalid, we skipped the record. Otherwise, we added the record to the `cleaned_data` list. The function returns this `cleaned_data` list which is a list of records with only valid performance ratings.

4. Fixing Hire Date Format

The issue was that some hire dates were not in a consistent format. Some were written as DD/MM/YYYY instead of the required YYYY-MM-DD format. This is a problem because inconsistent date formats make it difficult to compare or analyse dates correctly.

To fix this, we used a for loop to go through each employee record. We accessed the hire date at index 9 and checked if the date contained two `"/"` symbols using `.count('/')`.

If the format was incorrect, we used `.split('/')` to separate the date into day, month, and year. We then rearranged it into the correct format using an f-string:
`YYYY-MM-DD`

This updated the original record directly (in-place modification). If the date was already in the correct format, we left it unchanged.

5. Removing Invalid Dates

The issue was that some hire dates were logically invalid, even if they were in the correct format. This is a problem because incorrect dates would affect any time-based analysis in the dataset.

To fix this, we used a for loop to iterate through each record and extracted the hire date at index 9. We then split the date using `-` and converted the values into integers so we could perform checks.

We created an empty list called `cleaned_data` to store only valid records.

We then applied several validation rules:

- The year must be between 2015 and 2025
- The month must be between 1 and 12
- The day must be valid for that specific month
- February uses leap year logic to decide if it has 28 or 29 days
- Months like April, June, September, and November have 30 days

If any condition was not met, we used `continue` to skip that record.

If all checks passed, we appended the record to `cleaned_data`.

The function returns the final cleaned dataset containing only valid hire dates.

Section 2: Metrics

1. Unique Departments

We used a for loop to go through each employee record and extracted the department at index 1. We stored these in a set because sets automatically remove duplicates. The function returns all unique department names in the dataset.

2. Gender Distribution by Department

We created an empty dictionary called `gender_dist` to store a dictionary of gender counts in each department.

We used a for loop to iterate through the dataset and:

- extracted department (index 1)
- extracted gender (index 2)

If a department did not exist in the dictionary yet, we created an empty dictionary for it.

If a gender did not exist under that department, we set its count to 0.

We then incremented the count for each gender.

After counting, we used another nested loop to convert counts into percentages:

- total per department was calculated using `sum()`
- each gender count was divided by total and multiplied by 100
- results were rounded to 2 decimal places

The function returns a nested dictionary of percentages of genders in each department.

3. Average Age by Department

We created an empty dictionary called `dept_ages` where each department stores a list of ages.

Using a for loop, we grouped ages by department:

- if department did not exist, we created an empty list
- then appended age values (index 3)

After grouping, we used another for loop to calculate averages:

- `sum of list / length of list`
- rounded to 2 decimal places

The function returns a dictionary of average ages per department.

4. Retention Rate

We created a counter variable `status_active = 0` to track active employees.

We used a for loop to iterate through each record and checked status at index 8. If status was "Active", we incremented the counter.

We then divided active employees by total dataset size (`len(data)`) and multiplied by 100 to get the retention rate which we then rounded to 2 decimal places.

The function returns retention rate.

5. Turnover Rate by Department

We created a dictionary called `dept_counts` to store the total count of employees and the count of resigned employees.

We used a for loop to iterate through each record:

- incremented total employees per department
- if status was "Resigned", incremented resigned count

After building counts, we used another for loop to calculate turnover:

- $(\text{resigned} / \text{total}) \times 100$
- rounded to 2 decimal places

The function returns a dictionary of turnover rates per department.

6. Average Salary by Age Range

We created two variables:

- `total_salary = 0`
- `count = 0`

We used a for loop to filter employees within the given age range. For each match:

- salary was added to total
- counter was incremented

After looping, we calculated:

- `average = total_salary / count`

We also included a safety check:

- if `count = 0`, return 0 to avoid division errors

The result is rounded to 2 decimal places and returned as a float.

7. Average Department Performance by Training Hours Range

We first called `get_unique_departments(data)` to get all the departments.

We then created two dictionaries:

- rating_totals to store the sum of ratings per department
- matching_count to store stores number of valid employees per department

We used a for loop to filter employees based on training hours. For valid records:

- we added rating (index 6) to total
- incremented count

Finally, we used another for loop to compute averages:

- if no matches, set value to 0
- otherwise divide total / count
- round to 2 decimal places

The function returns a dictionary of average performance per department.

Conclusion

The project took us through the process of data analysis using Python. We were able to read the data using the helper function, which converted the dataset into a 2D list. We then cleaned the data by using functions to remove invalid entries and fix formatting issues. After that, we used the cleaned dataset to calculate HR metrics such as retention, turnover, and averages across departments. Overall, this project helped us turn raw employee data into meaningful insights.